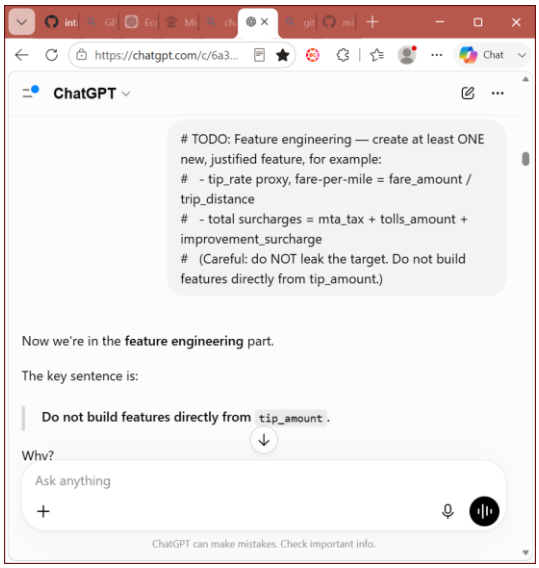
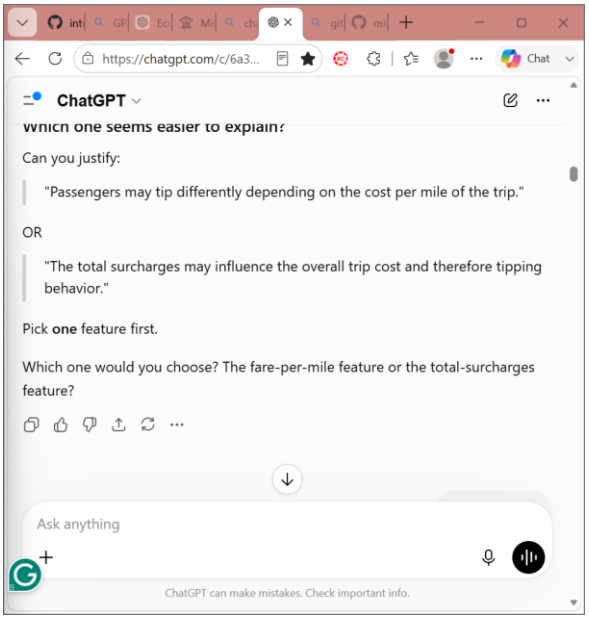
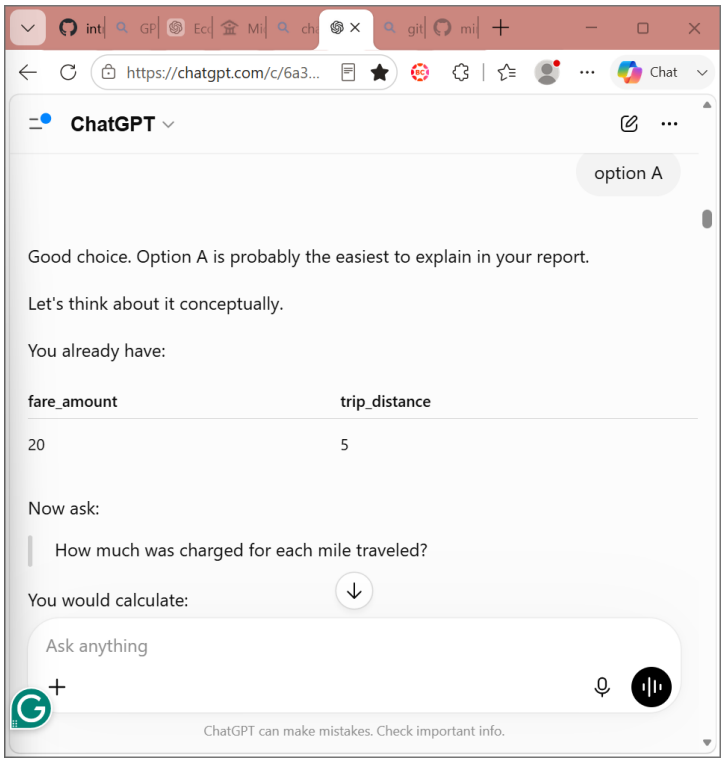
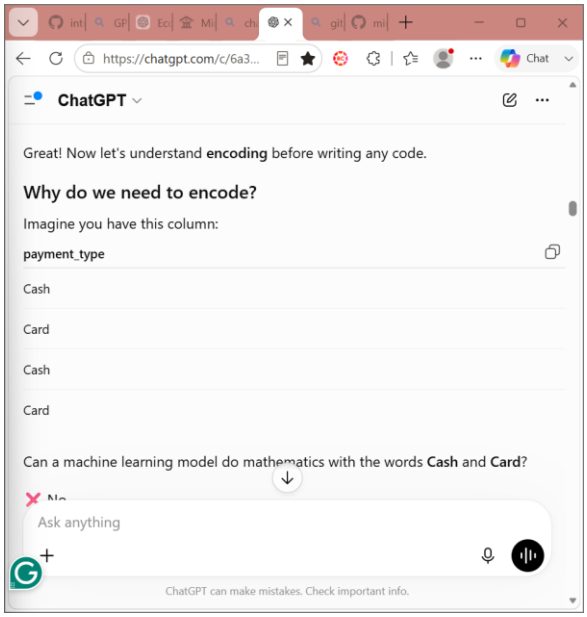
Preprocessing:

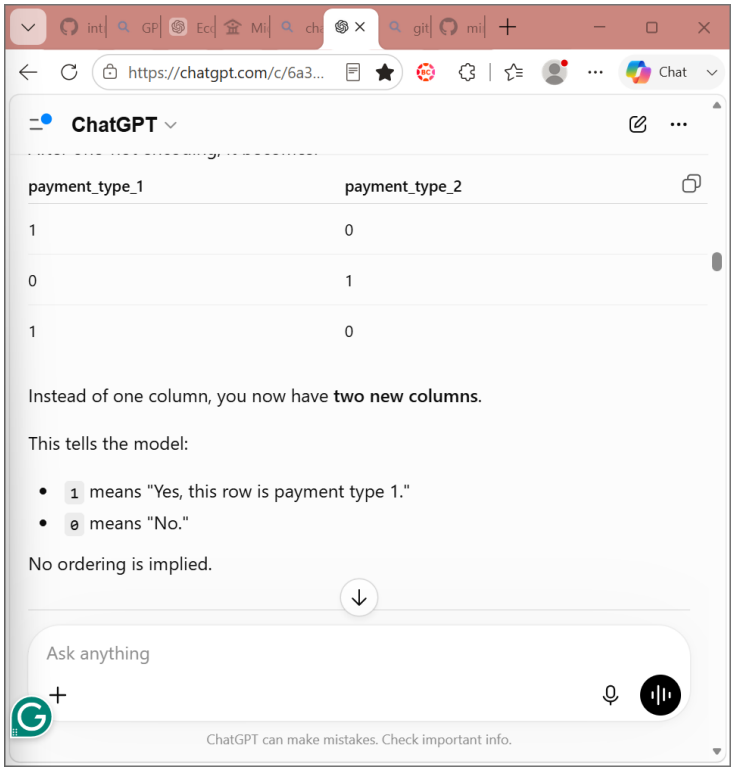


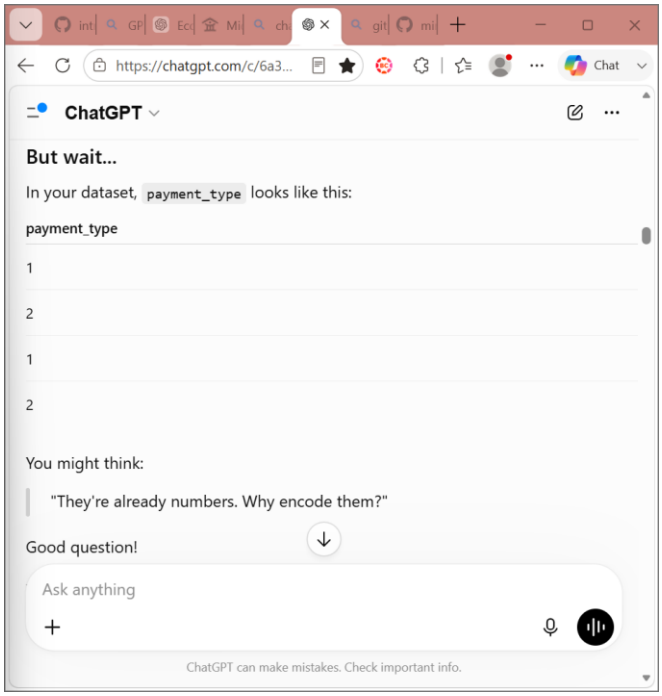


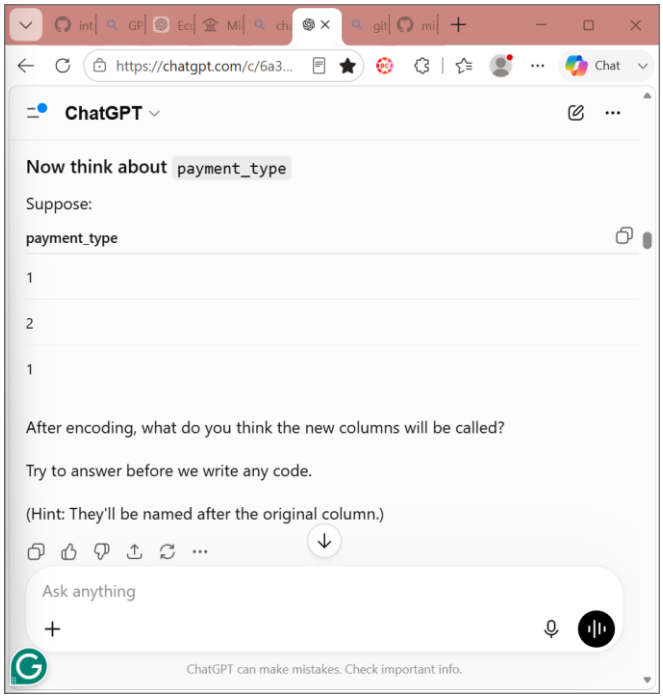


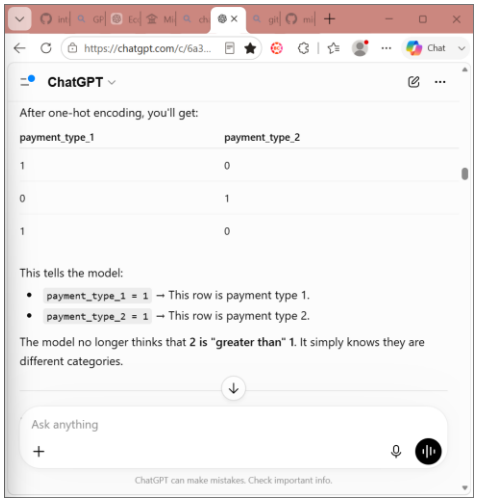
Encoding:

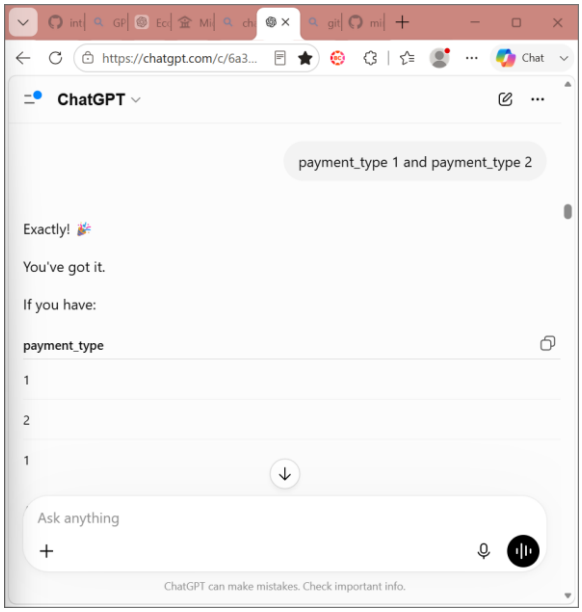

ChatGPT
Great! Now let's understand encoding before writing any code.
Why do we need to encode?
Imagine you have this column:
payment_type
Cash
Card
Cash
Card
Can a machine learning model do mathematics with the words Cash and Card?
Ask anything
ChatGPT can make mistakes. Check important info.

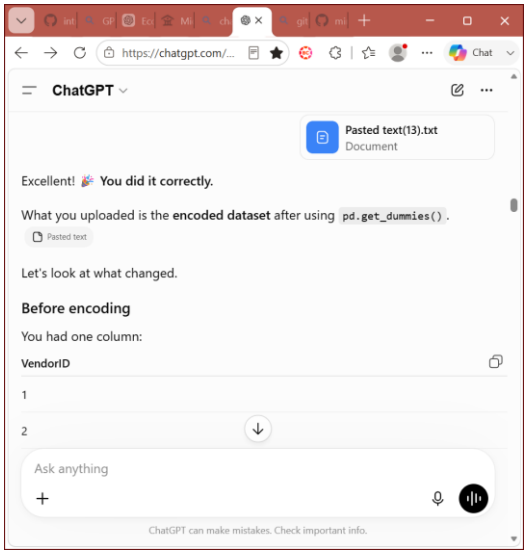

ChatGPT
payment_type_1
payment_type_2
1
0
0
1
1
0
Instead of one column, you now have two new columns.
This tells the model:
1 means "Yes, this row is payment type 1."
0 means "No."
No ordering is implied.
Ask anything
ChatGPT can make mistakes. Check important info.

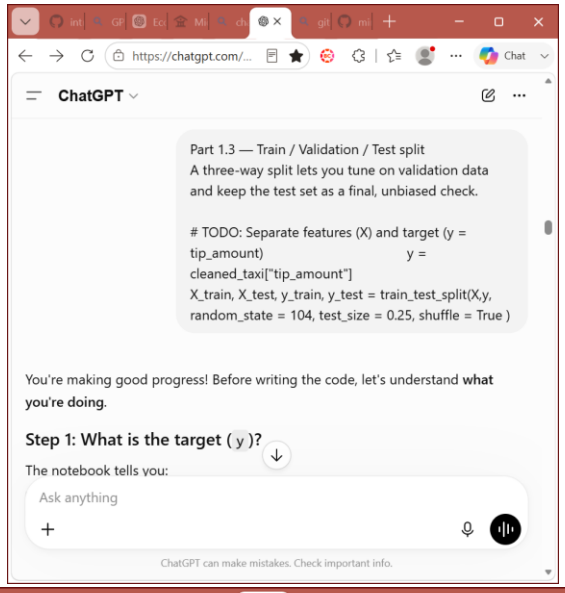

https://chatgpt.com/c/6a3...
Chat
ChatGPT
But wait...
In your dataset, payment_type looks like this:
payment_type
1
2
1
2
You might think:
"They're already numbers. Why encode them?"
Good question!
Ask anything
ChatGPT can make mistakes. Check important info.

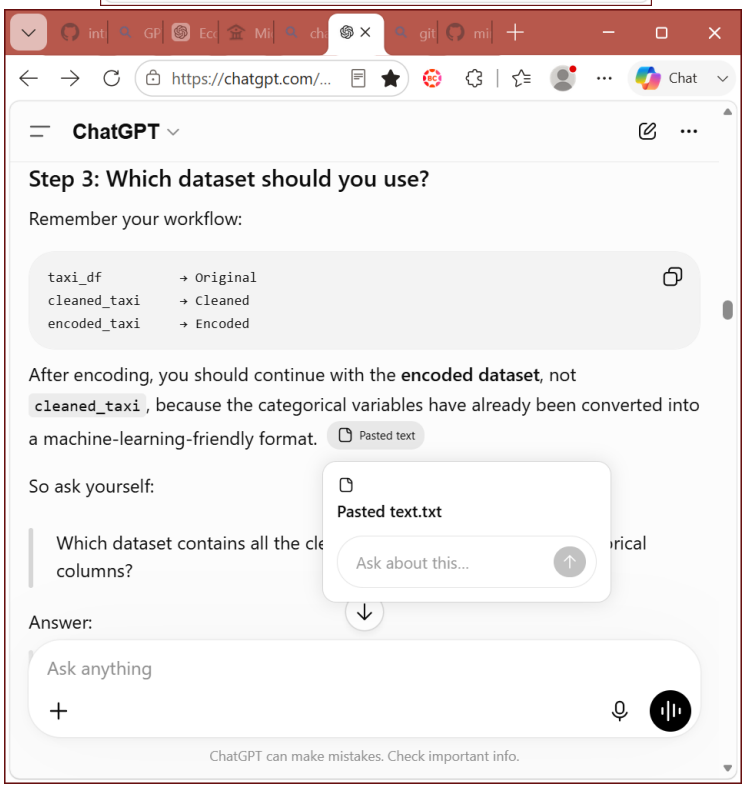

https://chatgpt.com/c/6a3...
Chat
ChatGPT
Now think about payment_type
Suppose:
payment_type
1
2
1
After encoding, what do you think the new columns will be called?
Try to answer before we write any code.
(Hint: They'll be named after the original column.)
Ask anything
ChatGPT can make mistakes. Check important info.

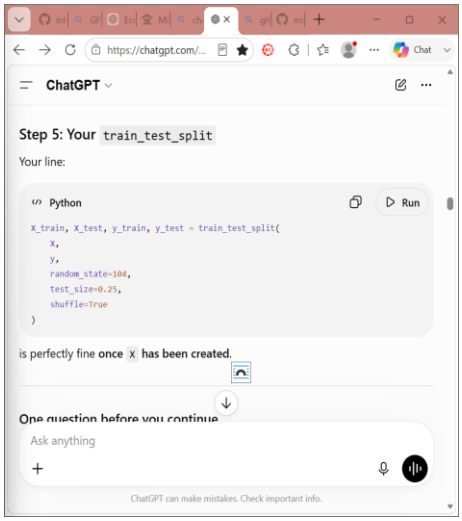

ChatGPT
After one-hot encoding, you'll get:
payment_type_1
payment_type_2
1
0
0
1
1
0
This tells the model:
payment_type_1 = 1 → This row is payment type 1.
payment_type_2 = 1 → This row is payment type 2.
The model no longer thinks that 2 is "greater than" 1. It simply knows they are different categories.
Ask anything
ChatGPT can make mistakes. Check important info.

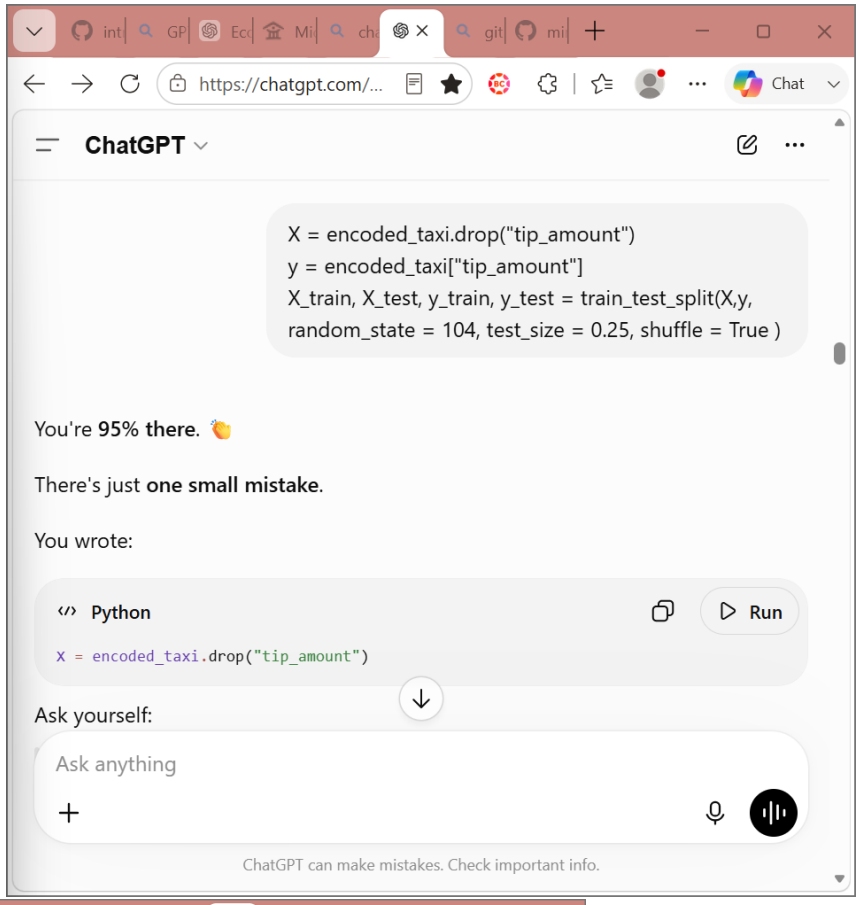

ChatGPT
payment_type 1 and payment_type 2
Exactly!
You've got it.
If you have:
payment_type
1
2
1
Ask anything
ChatGPT can make mistakes. Check important info.

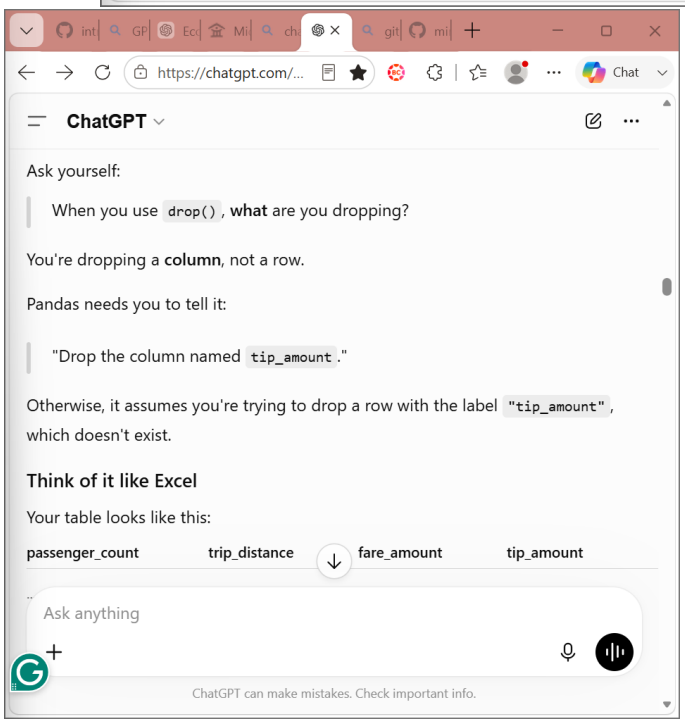

ChatGPT
Pasted text(13).txt
Document
Excellent! You did it correctly.
What you uploaded is the encoded dataset after using pd.get_dummies().
Pasted text
Let's look at what changed.
Before encoding
You had one column:
VendorID
1
2
Ask anything
ChatGPT can make mistakes. Check important info.

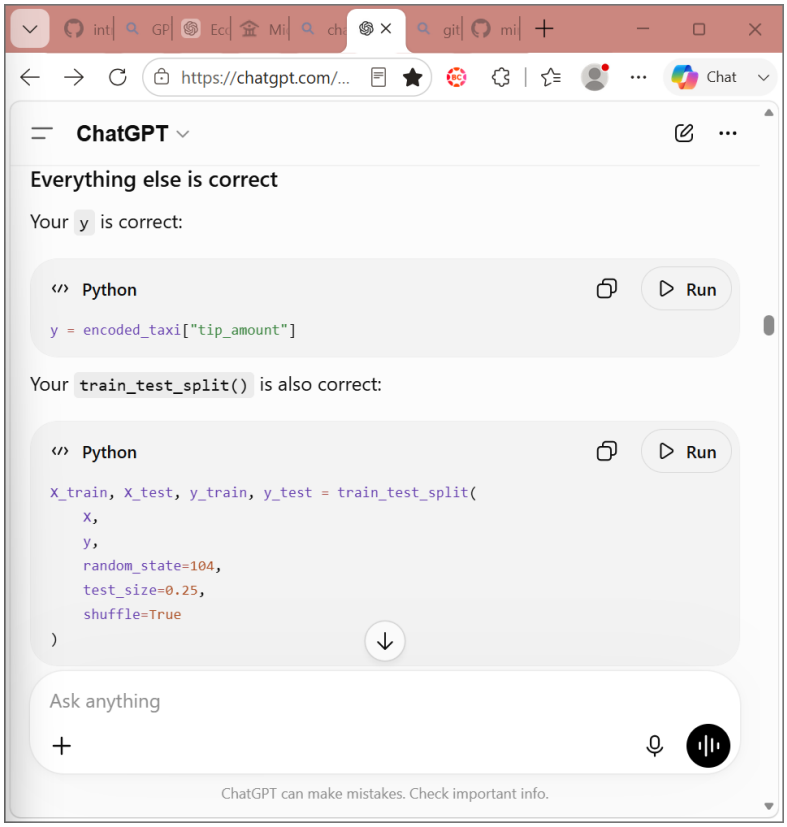

https://chatgpt.com/...
Chat
ChatGPT
Part 1.3 — Train / Validation / Test split
A three-way split lets you tune on validation data and keep the test set as a final, unbiased check.
TODO: Separate features (X) and target (y = tip_amount) y = cleaned_taxi["tip_amount"]
X_train, X_test, y_train, y_test = train_test_split(X,y, random_state = 104, test_size = 0.25, shuffle = True)
You're making good progress! Before writing the code, let's understand what you're doing.
Step 1: What is the target (y)?
The notebook tells you:
Ask anything
ChatGPT can make mistakes. Check important info.

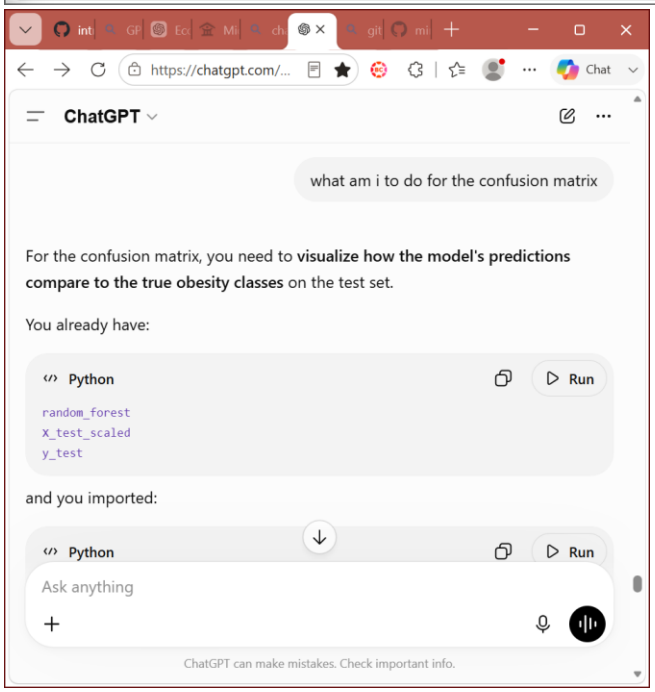

https://chatgpt.com/...
Chat
ChatGPT
Step 3: Which dataset should you use?
Remember your workflow:
taxi_df → Original
cleaned_taxi → Cleaned
encoded_taxi → Encoded
After encoding, you should continue with the encoded dataset, not cleaned_taxi, because the categorical variables have already been converted into a machine-learning-friendly format.
Pasted text
So ask yourself:
Pasted text.txt
Ask about this...
Which dataset contains all the cl rical columns?
Answer:
Ask anything
ChatGPT can make mistakes. Check important info.

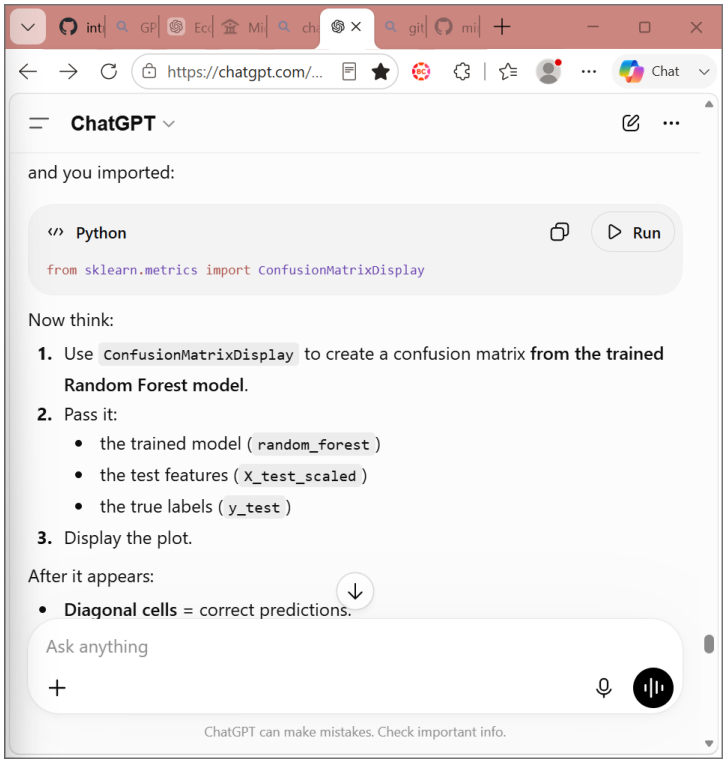

https://chatgpt.com/...
Chat
ChatGPT
Step 5: Your train_test_split
Your line:
Python
Run
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
random_state=104,
test_size=0.25,
shuffle=True
)
is perfectly fine once X has been created.
One question before you continue
Ask anything
ChatGPT can make mistakes. Check important info.

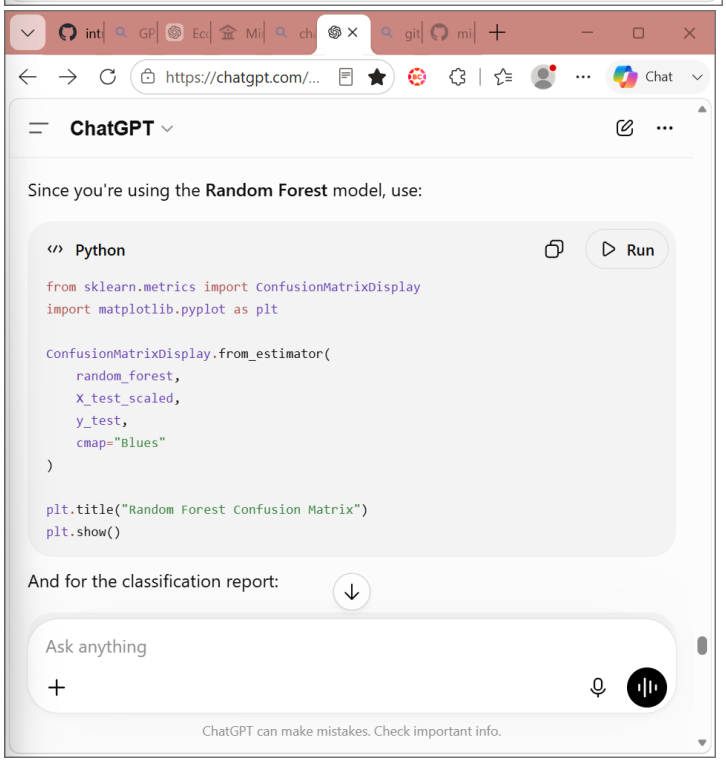

ChatGPT
X = encoded_taxi.drop("tip_amount")
y = encoded_taxi["tip_amount"]
X_train, X_test, y_train, y_test = train_test_split(X,y,
random_state = 104, test_size = 0.25, shuffle = True)
You're 95% there.
There's just one small mistake.
You wrote:
Python
Run
X = encoded_taxi.drop("tip_amount")
Ask yourself:
Ask anything
ChatGPT can make mistakes. Check important info.

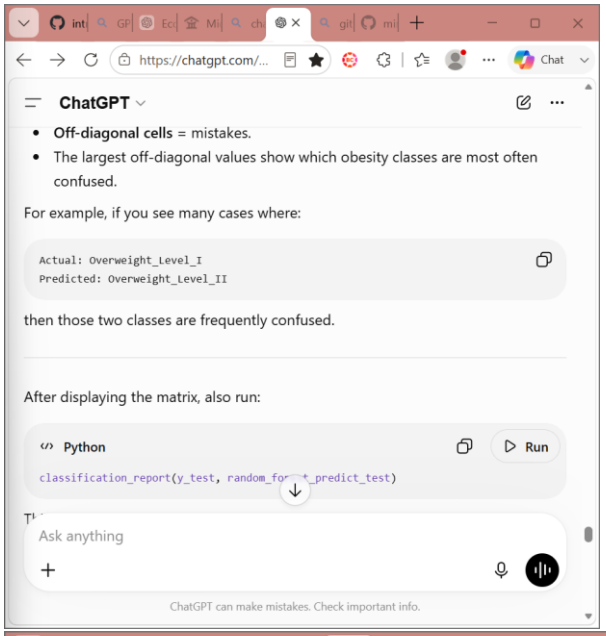

ChatGPT
Ask yourself:
When you use drop(), what are you dropping?
You're dropping a column, not a row.
Pandas needs you to tell it:
"Drop the column named tip_amount."
Otherwise, it assumes you're trying to drop a row with the label "tip_amount", which doesn't exist.
Think of it like Excel
Your table looks like this:
passenger_count
trip_distance
fare_amount
tip_amount
Ask anything
ChatGPT can make mistakes. Check important info.

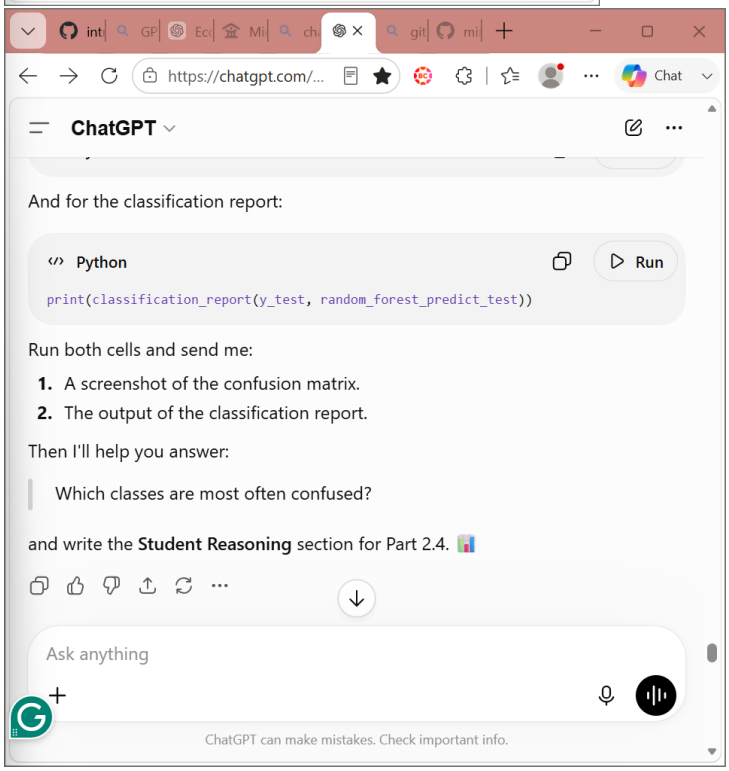

https://chatgpt.com/...
Chat
ChatGPT
Everything else is correct
Your y is correct:
Python
Run
y = encoded_taxi["tip_amount"]
Your train_test_split() is also correct:
Python
Run
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
random_state=104,
test_size=0.25,
shuffle=True
)
Ask anything
ChatGPT can make mistakes. Check important info.


https://chatgpt.com/...
Chat
ChatGPT
what am i to do for the confusion matrix
For the confusion matrix, you need to visualize how the model's predictions compare to the true obesity classes on the test set.
You already have:
Python
Run
random_forest
X_test_scaled
y_test
and you imported:
Python
Run
Ask anything
ChatGPT can make mistakes. Check important info.


https://chatgpt.com/...
Chat
ChatGPT
and you imported:
Python
Run
from sklearn.metrics import ConfusionMatrixDisplay
Now think:
1. Use ConfusionMatrixDisplay to create a confusion matrix from the trained Random Forest model.
2. Pass it:
• the trained model (random_forest)
• the test features (X_test_scaled)
• the true labels (y_test)
3. Display the plot.
After it appears:
• Diagonal cells = correct predictions.
Ask anything
ChatGPT can make mistakes. Check important info.


https://chatgpt.com/...
Chat
ChatGPT
Since you're using the Random Forest model, use:
Python
Run
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt
ConfusionMatrixDisplay.from_estimator(
random_forest,
X_test_scaled,
y_test,
cmap="Blues"
)
plt.title("Random Forest Confusion Matrix")
plt.show()
And for the classification report:
Ask anything
ChatGPT can make mistakes. Check important info.


https://chatgpt.com/...
Chat
ChatGPT
Off-diagonal cells = mistakes.
The largest off-diagonal values show which obesity classes are most often confused.
For example, if you see many cases where:
Actual: Overweight_Level_I
Predicted: Overweight_Level_II
then those two classes are frequently confused.
After displaying the matrix, also run:
Python
Run
classification_report(y_test, random_fo t_predict_test)
Ask anything
ChatGPT can make mistakes. Check important info.


https://chatgpt.com/...
Chat
ChatGPT
And for the classification report:
Python
Run
print(classification_report(y_test, random_forest_predict_test))
Run both cells and send me:
1. A screenshot of the confusion matrix.
2. The output of the classification report.
Then I'll help you answer:
Which classes are most often confused?
and write the Student Reasoning section for Part 2.4.
Ask anything
ChatGPT can make mistakes. Check important info.